

Programmation 1 - Examen

Durée : 2 heures.

Les documents manuscrits et dactylographiés sont autorisés. Les exercices sont indépendants. La clarté et la précision de la rédaction seront un facteur important de l'évaluation. Le barème est donné à titre indicatif.

Exercice 1 (12 points)

On considère, dans cet exercice, des communes françaises en utilisant les classes suivantes :

```
public class Departement extends Table<String, String>
// les communes d'un département
début
/* une clef de la table est le numéro INSEE d'une commune
la valeur est nom de la commune correspondante. Par exemple :
("35001", "Acigné")
("35238", "Rennes")
*/
...
fin
```

```
public class Communes extends Table<String, Departement>
début
/* une clef de la table est le numéro du département
et la valeur correspondante est une table contenant les numéros INSEE
et les noms des communes de ce département. Par exemple :
("35", ({("35001", "Acigné"), ("35002", "Amanlis")})
("56", ({("56001", "Allaire"), ("56002", "Ambron")})
*/
...
fin
```

N.B. Les numéros des départements et des communes sont des chaînes de caractères pour conserver les zéro initiaux éventuels.

On rappelle que la méthode `estTerminé()` de la classe `Iterateur` renvoie `vrai` quand l'itérateur a dépassé le dernier élément et que la méthode `oter()` de la classe `Iterateur` positionne l'itérateur sur l'élément qui suivait l'élément supprimé.

Écrire les fonctions suivantes.
Les réponses seront fournies à l'aide des méthodes `Table`.

Question 1

```
public static String nomDeCommune(Communes c, String dept, String insee)
// résultat : le nom de la commune du département dept et de numéro
// INSEE insee (null si absente).
```

Question 2

```
public static void afficherGrandsDepartements(Communes c, int min)
// effet : affiche à l'écran les numéros des départements possédant
// plus de min communes
```

Question 3

```
public static void supprimer(Communes c, String nomCommune)
// effet : toutes les communes de nom nomCommune sont supprimées de
// la table c.
```

Question 4

```
public static void miseAJour(Communes c, Communes maj)
// effet : effectue la mise à jour de la table c en utilisant la
// table maj (qui contient de nouvelles communes, ou des
// communes qui changent de nom).
```

Question 5

```
public static String nomLePlusRepandu(Communes c)
// pré-requis : !c.estVide()
// résultat : le nom de la commune qui apparaît le plus grand nombre
// de fois dans la table c.
```

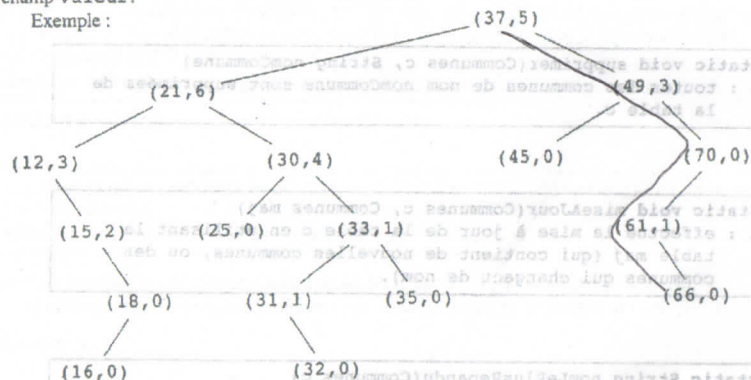
Exercice 2 (8 points)

On considère des arbres binaires de doublets déclarés par :

```
public class Doublet
débüt
    public int valeur, nombreDroite;
    public Doublet () début valeur = 0; nombreDroite = 0; fin
    public Doublet (int valeur, int nombreDroite)
    début
        this.valeur = valeur; this.nombreDroite = nombreDroite;
    fin
    ...
fin // class Doublet
```

Pour tout nœud d'un arbre de doublets, *valeur* est une valeur entière positive qui caractérise le nœud et *nombreDroite* est le nombre de nœuds figurant dans le sous-arbre droit. Tous les nœuds d'un arbre ont des valeurs *valeur* distinctes. L'arbre est organisé pour une recherche symétrique relativement au champ *valeur*.

Exemple :



Les réponses seront fournies à l'aide des méthodes *ArbreBinaire*.

Question 1

```
public static boolean present(ArbreBinaire<Doublet> A, int deb, int fin)
// pré-requis : !A.estVide()
// résultat : vrai si l'arbre contient un doublet dont la valeur est
//             comprise entre deb et fin (bornes incluses)
//             faux sinon.
```

Question 2

```
public static boolean linéaire(ArbreBinaire<Doublet> A)
// pré-requis : !A.estVide()
// résultat : vrai si A ne comporte qu'une seule branche
//             (c'est-à-dire il n'y a pas de nœud double dans A),
//             faux sinon.
```

Question 3

On appellera le *poids* d'une branche d'un arbre de doublets, la somme des valeurs des doublets se trouvant sur cette branche. Le *poids* de l'arbre est le plus grand poids parmi les branches.

```
public static int poids(ArbreBinaire<Doublet> A)
// pré-requis : !A.estVide()
// résultat : le poids de l'arbre A (283 pour l'exemple ci-dessus).
```

Question 4

```
public static void ajouter(ArbreBinaire<Doublet> A, Doublet d)
// pré-requis : !A.estVide()
// effet : le doublet d est rajouté à l'arbre (sans faire appel à la
//         fonction present()) et les champs nombreDroite sont mis à
//         jour quand c'est nécessaire.
//         Sans effet si un doublet de valeur d.valeur déjà présent.
//         Par exemple ajouter 60 dans l'arbre ci-dessus donne :
```

